Formal Modeling, Verification and Extended Implementation of a Secure ATM Transaction System using Automata Theory

Team Members

This project has been designed and developed by:

- Trinabha Dixit - 24BYB0026
- Bhargav Mahanta - 24BYB0046

The team collaboratively worked on conceptual modeling, interface design, implementation of automata logic, and integration of theoretical concepts with real-world ATM behavior.

Aim

The primary objective of this project is to model, simulate, and verify the functioning of an ATM transaction system using core concepts from Theory of Computation (TOC). The project aims to demonstrate how computational models such as finite automata, grammars, and machines can be applied to real-world systems.

The system is designed not only to simulate ATM operations but also to validate transaction sequences formally. By integrating multiple TOC concepts into a single system, the project provides a deeper understanding of how abstract mathematical models can be used to design secure, structured, and predictable systems.

System Overview

The project represents an ATM system as a formal computational model. The system begins in an idle state and progresses through authentication, operation selection, transaction processing, and exit stages.

The workflow is governed by defined states and transitions, ensuring that only valid sequences of inputs are accepted. Invalid inputs lead to rejection states, thereby maintaining system correctness and security.

Key characteristics of the system include:

- Deterministic flow for valid transactions
- Controlled handling of invalid inputs
- Security enforcement through lock and timeout mechanisms
- Real-time visualization of automata behavior

Modules

The project is divided into multiple modules, each corresponding to a specific TOC concept and its practical implementation.

Module 1:- DFA-Based ATM Simulation

This module forms the backbone of the system and models the ATM workflow using a Deterministic Finite Automaton (DFA). Each state represents a stage in the ATM process, and transitions occur based on user inputs.

The system ensures that for every input symbol, there is exactly one defined transition, maintaining determinism.

States involved:

- q0 – Idle state
- q1 – Authentication state
- q2 – Menu selection
- q3 – Processing state
- q4 – Cash dispensing
- q_lock – Locked state
- q_dead – Dead (trap) state

TOC Concepts Used:

- Deterministic Finite Automaton (DFA)
- Transition function $\delta(q, a)$

- Accepting and rejecting states
- Trap state concept

Module 2:- NFA and ε-NFA Extension

This module extends the DFA model by introducing Non-Deterministic Finite Automata (NFA) behavior. In this context, a single input (such as PIN entry) can lead to multiple possible outcomes.

Additionally, ε-transitions are used to represent automatic internal movements without consuming input symbols.

Applications in the project:

- Multiple outcomes for PIN input (success, retry, lock, failure)
- Internal transitions without explicit user input

TOC Concepts Used:

- Non-Deterministic Finite Automaton (NFA)
- ε-NFA (epsilon transitions)
- Set of possible next states
- Transition ambiguity

Module 3:- Regular Expression and Context-Free Grammar

This module defines the ATM transaction language formally using both Regular Expressions and Context-Free Grammar (CFG).

The valid transaction sequence is represented as:

- Regular Expression:
  (card)(pin)(op)(withdraw)(exit)
- CFG Representation:
  - S → card A
  - A → pin B
  - B → op C
  - C → withdraw D
  - D → exit

The system includes a regex validation tool to verify whether a given input string belongs to the defined language.

TOC Concepts Used:

- Regular Languages
- Regular Expressions
- Context-Free Grammar (CFG)
- Language generation and validation

Module 4:- Pushdown Automata (PDA) Simulation

This module introduces memory into the system using a stack-based Pushdown Automaton. It demonstrates how additional computational power can be achieved beyond finite automata.

The stack is used to track operations such as card insertion and PIN verification.

Operations performed:

- Push symbols onto the stack during input
- Pop symbols during exit or reset
- Display instantaneous description (ID)

TOC Concepts Used:

- Pushdown Automaton (PDA)
- Stack operations (push/pop)
- Instantaneous Description (ID)
- Context-free language recognition

Module 5:- Turing Machine Simulation

This module represents the ATM system using a Turing Machine, the most powerful computational model.

The system uses a tape and a movable head to simulate computation and verify transaction validity.

Functionality:

- Reads input from tape
- Moves head left/right
- Verifies balance vs withdrawal
- Produces accept/reject output

TOC Concepts Used:

- Turing Machine (TM)
- Tape and head movement
- Decidability
- Halting condition

Module 6:- Security and System Enhancements

This module integrates real-world ATM security features into the computational model, making the system more practical and realistic.

Security features include:

- Lock state after three incorrect PIN attempts
- Automatic timeout reset to initial state
- Dead state for invalid input sequences
- Transaction logging with timestamps

TOC Concepts Used:

- Trap states
- Error handling via rejection states
- System recovery transitions
- Practical extension of formal models

Highlights / Features

The project provides a comprehensive and interactive platform for understanding Theory of Computation concepts through a real-world application.

Key highlights include:

- Interactive ATM simulation with real-time state transitions
- Graphical visualization of automata

- Integration of DFA, NFA, CFG, PDA, and Turing Machine
- Built-in validation tools for input sequences
- Security mechanisms like lock state and timeout
- Logging system for tracking transitions

GitHub Repository Link- For Code

👉 https://github.com/ItsTrinabha/secure-atm-automata

Website Link

👉 https://tocproject-one.vercel.app/

Screenshots and Demonstration

To validate the correctness and functionality of the ATM automata system, multiple execution scenarios were tested and documented using screenshots. Each case demonstrates a specific system behavior and is supported with a corresponding image. The screenshots are placed immediately after each case description to maintain clarity and continuity between theoretical explanation and practical execution.

Case 1: Successful Transaction

In this scenario, the user follows the correct sequence of operations: card insertion, correct PIN entry, operation selection, withdrawal, and exit. The system transitions through states q0 → q1 → q2 → q3 → q4 → q0, completing the transaction successfully. This confirms that the input string belongs to the accepted language of the automaton.

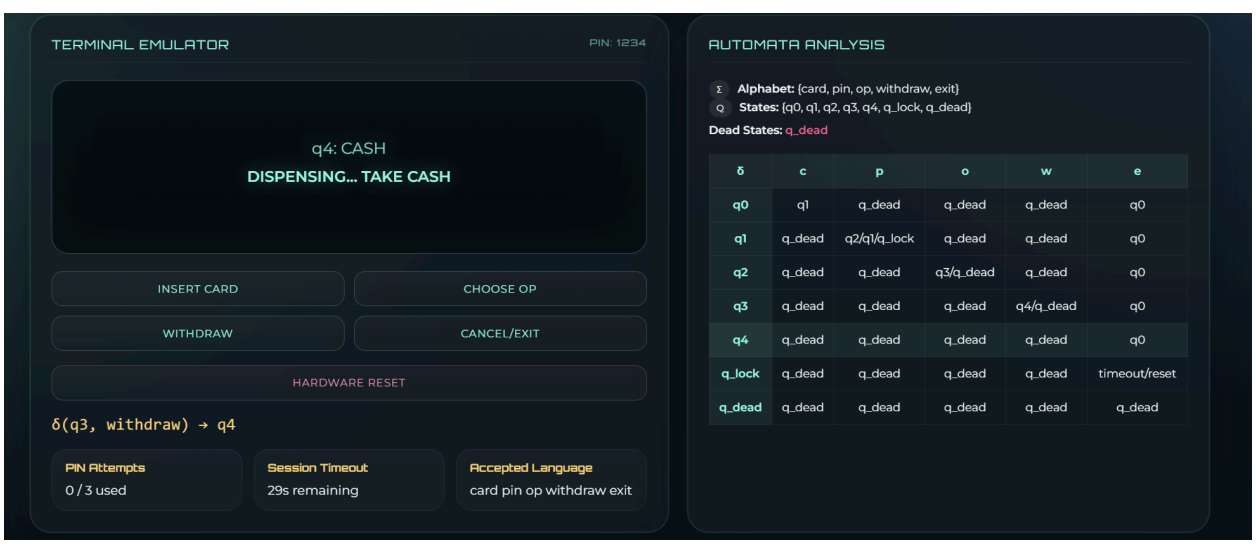


Figure 1: Successful transaction showing valid state transitions from q0 to q4 and returning to q0 after completion.

Case 2: Incorrect PIN Attempt

In this case, the user enters an incorrect PIN during the authentication phase. The system remains in state q1 and increments the attempt counter, allowing the user to retry within the allowed limit. This demonstrates controlled error handling without immediate rejection.

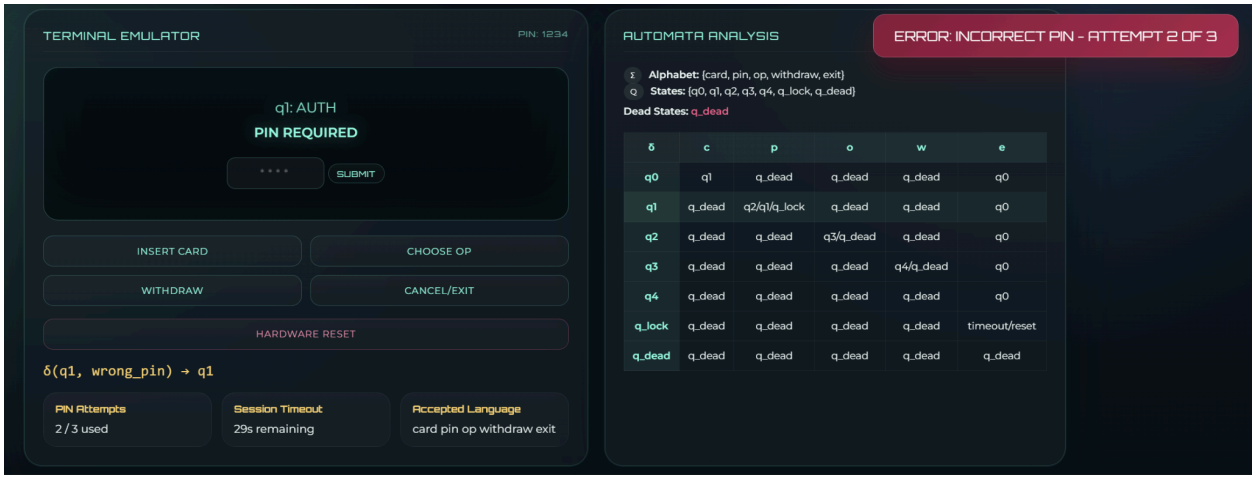


Figure 2: Incorrect PIN entry showing retry behavior where the system remains in authentication state q1.

Case 3: Account Lock (q_lock)

After three consecutive incorrect PIN attempts, the system transitions to the locked state q_lock. This state prevents further interaction, ensuring account security and preventing unauthorized access.

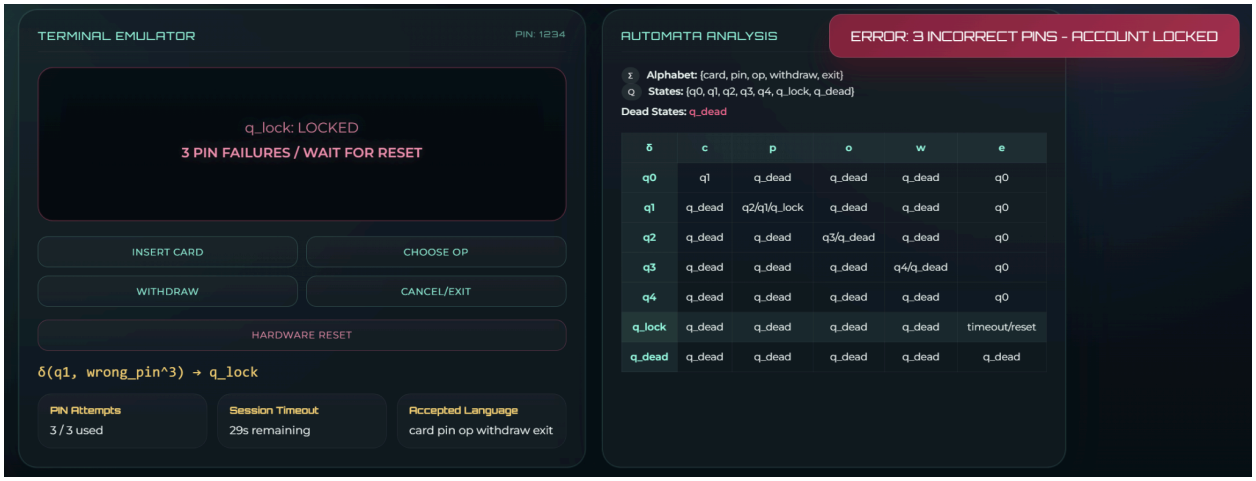
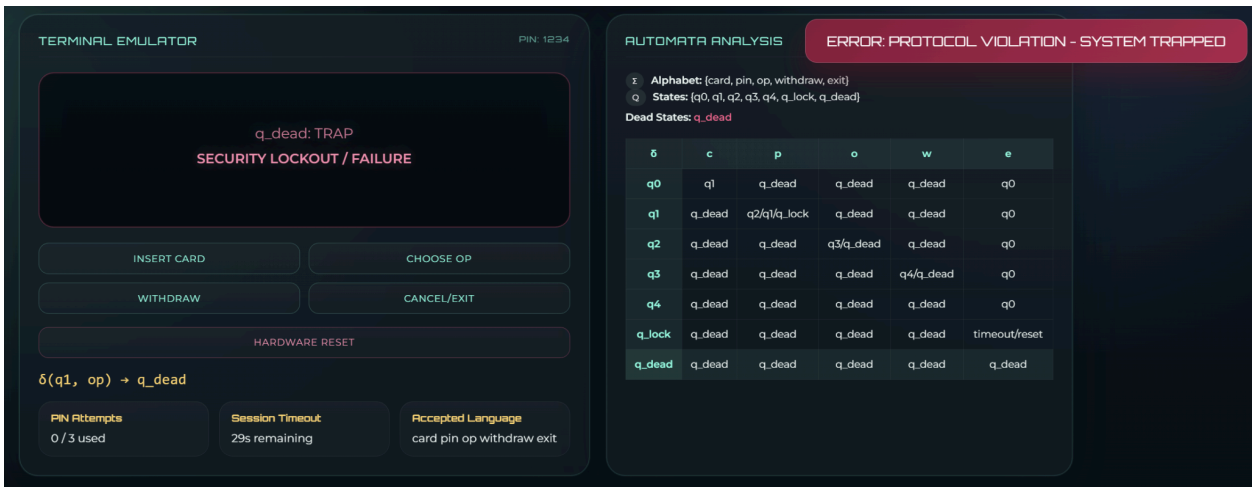


Figure 3: System enters locked state q_lock after three consecutive incorrect PIN attempts, preventing further interaction.

Case 4: Protocol Violation (q_dead)

This case demonstrates an invalid sequence of operations, such as attempting to withdraw money before authentication. The system immediately transitions to the trap state q_dead, indicating a violation of the defined transaction protocol.

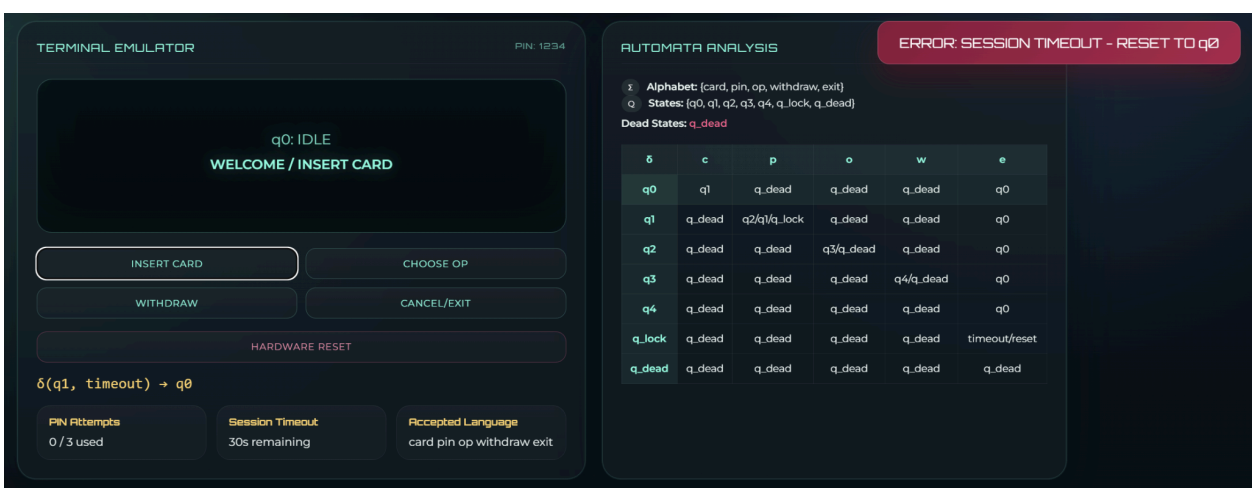


δ	c	p	o	w	e
q0	q1	q_dead	q_dead	q_dead	q0
q1	q_dead	q2/q1/q_lock	q_dead	q_dead	q0
q2	q_dead	q_dead	q3/q_dead	q_dead	q0
q3	q_dead	q_dead	q_dead	q4/q_dead	q0
q4	q_dead	q_dead	q_dead	q_dead	q0
q_lock	q_dead	q_dead	q_dead	q_dead	timeout/reset
q_dead	q_dead	q_dead	q_dead	q_dead	q_dead

Figure 4: Invalid operation sequence leading to trap state q_dead, demonstrating strict enforcement of transaction order.

Case 5: Timeout Reset

If the user remains inactive for a specified duration, the system automatically resets to the initial state q0. This prevents session misuse and ensures system safety.



δ	c	p	o	w	e
q0	q1	q_dead	q_dead	q_dead	q0
q1	q_dead	q2/q1/q_lock	q_dead	q_dead	q0
q2	q_dead	q_dead	q3/q_dead	q_dead	q0
q3	q_dead	q_dead	q_dead	q4/q_dead	q0
q4	q_dead	q_dead	q_dead	q_dead	q0
q_lock	q_dead	q_dead	q_dead	q_dead	timeout/reset
q_dead	q_dead	q_dead	q_dead	q_dead	q_dead

Figure 5: Automatic session timeout resetting the system to initial state q0 after inactivity

Case 6: Hardware Reset Recovery

When the system enters either q_dead or q_lock, the hardware reset function allows recovery by returning the system to the initial state q0. This ensures that the system can restart cleanly after an error or lock condition.

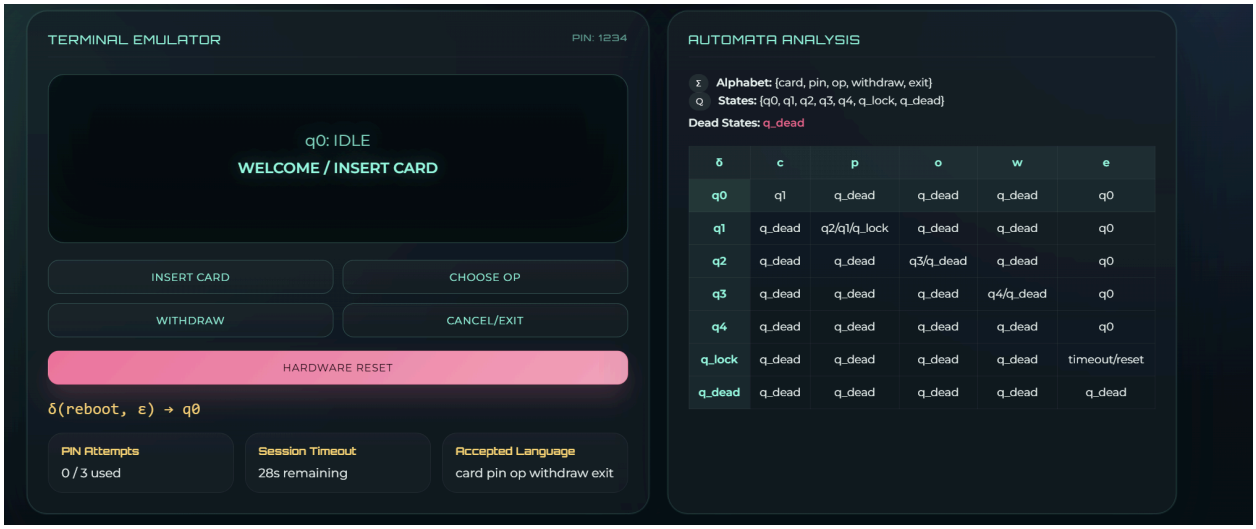


Figure 6: System recovery using hardware reset, restoring the automaton from error states to initial state q0.

Project Demonstration Video

https://drive.google.com/file/d/1mTn2sLzgAknKbc-3G-YR0iobsT1u9ZTh/view?usp=sharing

Automata Modeling

The ATM system is formally defined as a finite automaton M = (Q, Σ, δ, q0, F), where Q is the set of states, Σ is the input alphabet, δ is the transition function, q0 is the initial state, and F is the set of accepting states. In this model, Q = {q0, q1, q2, q3, q4, q_lock, q_dead}, Σ = {card, pin, op, withdraw, exit}, q0 denotes the idle state, and F = {q4}, representing successful transaction completion.

The modeling enforces a linear, protocol-driven flow of execution in which each state corresponds to a well-defined stage of the ATM lifecycle. The

semantics of each state are provided below to ground the formal definition in system behavior.

State	***Description***	***System Interpretation***
q0	**Idle**	**System awaits card insertion and user interaction**
q1	**Authentication**	**User enters and verifies PIN**
q2	**Menu**	**User selects operation (e.g., withdraw)**
q3	**Processing**	**System executes selected operation**
q4	**Completion**	**Cash dispensed; transaction successful**
q_lock	**Locked**	**Account locked after repeated failures**
q_dead	**Trap**	**Invalid sequence or security violation**

This mapping ensures that the DFA captures both correctness constraints and security invariants of the ATM workflow.

Mathematical Properties of the DFA

The constructed automaton satisfies the properties of a deterministic finite automaton. For every pair $(q, a) \in Q \times \Sigma$, the transition function $\delta(q, a)$ yields exactly one next state, ensuring determinism. The language accepted by the automaton is a regular language characterized by a fixed sequence of symbols.

The accepted language is defined as:

L(M) = { card · pin · op · withdraw · exit }

To ensure totality of δ, all undefined transitions are redirected to the trap state q_dead, making the DFA complete. The presence of q_dead guarantees closure under Σ and simplifies reasoning about rejection, since any invalid prefix or out-of-order symbol leads to a sink state with no recovery transitions (except via reset at the system level).

Transaction Validation and Workflow Analysis

The system enforces a strict protocol in which the user must follow the sequence card → pin → op → withdraw → exit. This sequence models a canonical ATM withdrawal session. Any deviation—such as attempting an operation before authentication, or invoking withdrawal without selecting an operation—constitutes a protocol violation and results in transition to q_dead.

Authentication is governed by a bounded retry policy. Incorrect PIN entries are permitted up to a threshold, after which the system transitions to q_lock to prevent brute-force attacks. Additionally, inactivity triggers a timeout transition that returns the system to q0, preventing stale sessions.

Input Condition	***System Behavior***	***Final State***
Correct sequence	**Transaction successful**	**q4**
Incorrect PIN (1–2 attempts)	**Retry allowed**	**q1**
Incorrect PIN (3 attempts)	**Account locked**	**q_lock**
Invalid sequence	**Immediate rejection**	**q_dead**
Timeout	**Session reset**	**q0**

This validation strategy ensures both safety (reject invalid inputs) and liveness (complete valid transactions).

DFA Transition Logic and Implementation

The transition function δ is implemented using a JavaScript object for efficient state dispatch. Each user action (input symbol) is mapped to a deterministic next state based on the current state. The implementation separates control flow (δ) from side effects (UI updates, logging), enabling clearer reasoning and easier maintenance.

```
const transitions = { q0: { card: "q1", exit: "q0" }, q1: { exit: "q0" }, q2: { op: "q3", exit: "q0" }, q3: { withdraw: "q4", exit: "q0" }, q4: { exit: "q0" } };
```

The operational semantics can be summarized in a transition table that aligns inputs with state changes and their meanings.

Current State	***Input Symbol***	***Next State***	***Meaning***
q0	**card**	**q1**	**Card inserted; begin session**
q1	**pin (correct)**	**q2**	**Authentication successful**
q2	**op**	**q3**	**Operation selected**
q3	**withdraw**	**q4**	**Withdrawal processed**
q4	**exit**	**q0**	**Session terminated**

This deterministic mapping ensures that for any valid prefix of the accepted language, the next state is uniquely determined.

Non-Deterministic Extensions (NFA Interpretation)

While the implemented system is deterministic, it admits an NFA-style interpretation during authentication. The input symbol "pin" can lead to

multiple conceptual outcomes depending on runtime conditions such as correctness and attempt count. This is captured as:

$\delta(q1, pin) = \{ q2, q1, q_lock, q_dead \}$

Here, q2 represents successful authentication, q1 represents retry on incorrect entry, q_lock represents account lock after exceeding attempts, and q_dead captures anomalous or bypass conditions. This abstraction highlights how non-determinism can model conditional branching inherent in real-world systems.

Epsilon Transitions and Internal Computation

The system can be extended with ε-transitions to represent internal computation steps that do not consume input symbols. For instance, once a correct PIN is validated, the system may transition from q1 to q2 without explicit user action. Modeling such behavior using ε-transitions clarifies the separation between user-driven inputs and system-driven state changes, and aligns the model with ε-NFA semantics.

Pushdown Automata (PDA) Extension

To model memory beyond finite-state capabilities, the system incorporates a conceptual stack similar to a PDA. The stack records intermediate markers such as CARD, AUTH, and TX, enabling representation of nested or staged operations and facilitating rollback semantics.

The instantaneous description is given by (q, w, stack), where q is the current state, w is the remaining input, and stack is the sequence of symbols stored. Push operations correspond to entering new stages, while pop operations correspond to undoing or completing stages. Although the ATM workflow is largely regular, the PDA abstraction demonstrates how memory augments computational expressiveness.

Turing Machine Model and Decidability

A Turing Machine abstraction is introduced to model decision problems such as verifying whether a withdrawal request is less than or equal to the available balance. In this formulation, the tape encodes inputs (e.g., balance

and request), the head scans and updates symbols, and the transition function governs computation until a halting state is reached.

This model illustrates decidability in the context of transaction feasibility and provides a bridge from finite and context-free models to general computation. It also enables discussion of halting conditions, where the machine either accepts (sufficient balance) or rejects (insufficient funds).

Software Architecture and System Design

The system is implemented using a layered architecture that separates concerns into presentation, logic, and visualization. The user interface layer is built with HTML and CSS, providing a simulated ATM console with input controls and status displays. The application logic layer is implemented in JavaScript and encapsulates the DFA transition logic, validation rules, timeout management, and logging. The visualization layer leverages the vis-network library to render the automaton graph dynamically, highlighting the current state and transitions in real time.

Layer	***Components***	***Function***
User Interface	**HTML, CSS**	**Renders ATM screen and controls**
Logic Layer	**JavaScript**	**Executes δ, validation, and security rules**
Visualization	**vis-network**	**Displays state graph and transitions**

This separation improves modularity, testability, and extensibility of the system.

Security Mechanisms and Reliability

The system integrates multiple security controls aligned with real ATM behavior. A bounded PIN attempt policy mitigates brute-force attacks,

transitioning to q_lock after repeated failures. A trap state q_dead enforces protocol correctness by rejecting invalid sequences. A session timeout resets inactive sessions to q0, reducing exposure to misuse. A hardware reset mechanism provides controlled recovery from error states, restoring the system to a known safe configuration.

These mechanisms collectively ensure confidentiality (protecting PIN entry), integrity (preventing invalid operations), and availability (recovering from faults), thereby enhancing overall system reliability.

Advantages / Benefits

This project effectively connects theoretical concepts with practical applications, making it highly beneficial for both academic and conceptual understanding.

Key advantages:

- Provides a real-world application of TOC concepts
- Enhances visualization of abstract computational models
- Covers multiple topics in a single integrated system
- Useful for academic presentations and viva examinations
- Demonstrates structured system design and validation

Drawbacks / Future Improvements

While the project successfully demonstrates theoretical concepts, it has certain limitations that can be addressed in future enhancements.

Current limitations:

- Simplified ATM functionality without real banking integration
- Limited operations (primarily withdrawal)
- No database or backend support
- Simulation-level security only

Future improvements:

- Integration with backend technologies (Node.js, MongoDB)
- Addition of features like balance inquiry and deposit

- Implementation of multi-user authentication
- Deployment as a web or mobile application
- Hardware integration using microcontrollers
- Inclusion of AI-based fraud detection

Societal Measures and Usage

This project contributes to the understanding of secure system design and promotes awareness about digital transaction systems.

It highlights the importance of authentication, structured workflows, and error handling in modern banking systems.

Applications include:

- Educational tool for Theory of Computation
- Demonstration model for ATM systems
- Learning platform for secure system design

The project also promotes awareness regarding safe ATM usage, secure transactions, and the importance of designing reliable computational systems.

Conclusion

This project successfully demonstrates the application of Automata Theory in modeling and implementing a secure ATM transaction system. By representing the transaction workflow using a Deterministic Finite Automaton (DFA), the system ensures that all operations follow a well-defined and strictly ordered sequence, thereby maintaining correctness and reliability.

The integration of advanced computational models such as Non-deterministic Finite Automata (NFA), Pushdown Automata (PDA), Context-Free Grammar (CFG), and Turing Machine (TM) concepts enhances the depth of the system. These extensions allow the representation of conditional behavior, memory-based operations, and decision-making processes, bridging the gap between theoretical computation and practical system design.

The implementation further incorporates essential security mechanisms, including PIN validation with limited attempts, trap states for invalid

sequences, session timeout handling, and hardware reset functionality. These features ensure that the system is robust, fault-tolerant, and aligned with real-world ATM behavior. The inclusion of graphical visualization, transition matrices, and execution logs provides clear insight into the internal functioning of the automaton.

Through this project, it is evident that formal computational models are not merely theoretical constructs but powerful tools for designing secure and reliable systems. The ATM automata system highlights how concepts from Theory of Computation can be effectively applied in real-world applications, particularly in domains where correctness, security, and predictability are critical.